

# QUEUELESS

## Virtual Queue Management System

FINAL MASTER SPECIFICATION — MongoDB / MERN Architecture

**FINAL ARCHITECTURE**  
React + Vite + Tailwind CSS  
Node.js + Express.js  
MongoDB Atlas + Mongoose  
JWT + bcrypt authentication  
Socket.IO real-time communication  
Vercel + Render/Railway deployment

| Item           | Decision                                    |
|----------------|---------------------------------------------|
| Project        | QueueLess — Virtual Queue Management System |
| Architecture   | MERN-style full-stack application           |
| Database       | MongoDB Atlas                               |
| ODM            | Mongoose                                    |
| Authentication | JWT + bcrypt                                |
| Realtime       | Socket.IO                                   |
| AI             | None                                        |
| Primary users  | Customer, Staff, Admin                      |

# Table of Contents

1. Project Overview
2. Problem Statement
3. Proposed Solution
4. Goals and Objectives
5. Target Users and Roles
6. Core Features
7. Complete User Flows
8. Technology Stack
9. System Architecture
10. Frontend Architecture
11. Backend Architecture
12. MongoDB Architecture
13. Collections and Schemas
14. Relationships and References
15. Authentication Architecture
16. Authorization and Role-Based Access
17. Queue State Machine
18. Token Generation
19. Queue Ordering and Position
20. Estimated Wait Time
21. Priority Queue
22. Concurrency and Atomicity
23. Socket.IO Real-Time Architecture
24. API Specification
25. UI/UX Screen Plan
26. Customer Dashboard
27. Staff Dashboard
28. Admin Dashboard
29. QR Queue Joining
30. Kiosk Mode
31. Public TV Display
32. Notifications
33. Validation and Error Handling
34. Security
35. Analytics
36. Project Folder Structure
37. Development Roadmap
38. Testing Strategy
39. Deployment
40. Environment Variables
41. Git Strategy
42. MVP vs Advanced Features
43. Resume Positioning
44. Future Scope
45. Definition of Done

## 46. Exact Build Order for Antigravity

# 1. Project Overview

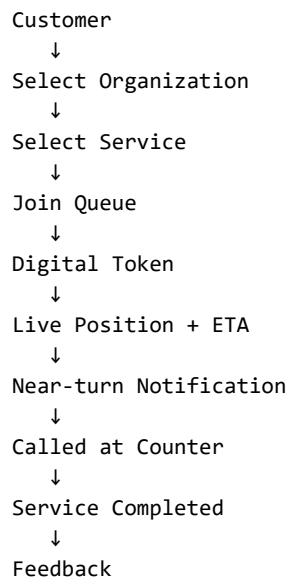
QueueLess is a full-stack virtual queue management platform. Customers can discover an organization and service, join a queue remotely, receive a digital token, track their position and estimated wait, and receive live updates when their turn approaches. Staff operate queues through a dedicated dashboard, while administrators configure organizations, services, counters, staff and analytics.

This version is intentionally AI-free. Its engineering depth comes from MERN-style full-stack development, MongoDB data modeling, authentication, role-based authorization, transactional/concurrency-aware queue operations, Socket.IO real-time communication, dashboards, QR entry, notifications and deployment.

## 2. Problem Statement

- Customers waste time physically standing in queues.
- Customers cannot easily see queue progress or estimated waiting time.
- Manual token systems can create skipped, duplicated or incorrectly served tickets.
- Multiple counters need coordinated queue operations.
- Organizations often lack historical queue and service analytics.

## 3. Proposed Solution



## 4. Goals and Objectives

- Digitize physical queues.
- Give customers visibility into token position and waiting time.
- Give staff reliable queue-control operations.
- Give admins complete operational control.
- Prevent duplicate active tickets and conflicting staff operations.
- Maintain an auditable history of queue states.
- Provide useful operational analytics.

- Create a production-style portfolio project without AI.

## 5. Target Users and Roles

| Role     | Purpose              | Main Capabilities                                                 |
|----------|----------------------|-------------------------------------------------------------------|
| Customer | Uses services        | Join/cancel queue, track ticket, notifications, history, feedback |
| Staff    | Operates queue       | Call next, recall, skip, no-show, serve, pause/resume             |
| Admin    | Manages organization | Services, counters, staff, queues, settings, analytics            |

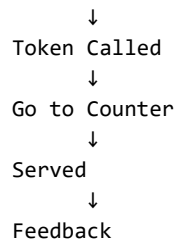
## 6. Core Features

| Feature                 | Description                                          |
|-------------------------|------------------------------------------------------|
| Authentication          | Signup/login/logout with JWT and bcrypt              |
| Role-based access       | Customer, staff and admin permissions                |
| Organization management | Organization profile and settings                    |
| Service management      | Services and average service duration                |
| Counter management      | Counters and staff assignments                       |
| Queue management        | Open/close/pause/resume queues                       |
| Digital token           | Server-generated unique queue token                  |
| Live position           | Current customers ahead                              |
| Estimated wait          | Deterministic ETA based on queue data                |
| Queue operations        | Call, recall, skip, no-show, serve                   |
| Realtime                | Socket.IO live updates                               |
| Notifications           | In-app queue alerts                                  |
| Analytics               | Volume, wait time, service time, completion, no-show |
| QR joining              | Scan QR to enter a service queue                     |
| Kiosk                   | Touch-friendly on-site joining                       |
| Public display          | TV-style current token/counter display               |
| Feedback                | Rating/comment after service                         |

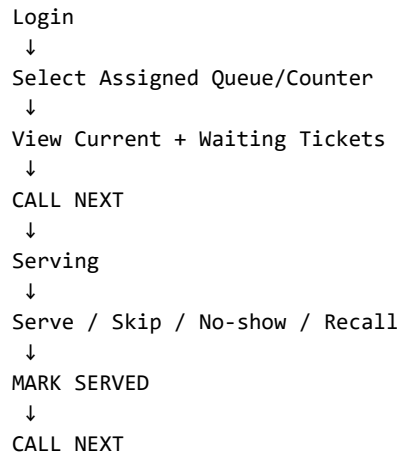
## 7. Complete User Flows

### 7.1 Customer

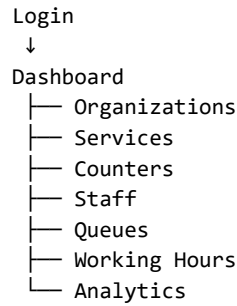
```
Login / Register
↓
Choose Organization
↓
Choose Service
↓
See Current Queue
↓
Join Queue
↓
Receive Token
↓
Track Position + ETA
↓
Near-turn Alert
```



7.2 Staff



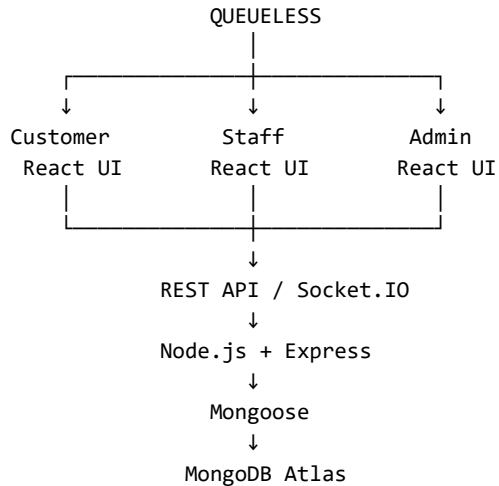
7.3 Admin



8. Technology Stack

| Layer          | Technology                              | Purpose                                         |
|----------------|-----------------------------------------|-------------------------------------------------|
| Frontend       | React.js + Vite                         | Application UI                                  |
| Styling        | Tailwind CSS                            | Responsive design                               |
| Routing        | React Router                            | Protected and role-based navigation             |
| HTTP           | Axios or fetch                          | API communication                               |
| Backend        | Node.js + Express.js                    | REST API and business logic                     |
| Database       | MongoDB Atlas                           | Cloud NoSQL database                            |
| ODM            | Mongoose                                | Schemas, validation, models and database access |
| Authentication | JWT + bcrypt                            | Secure login/session mechanism                  |
| Realtime       | Socket.IO                               | Live queue synchronization                      |
| Charts         | Recharts                                | Analytics dashboards                            |
| Icons          | Lucide React                            | UI icons                                        |
| QR             | qrcode                                  | QR generation                                   |
| Testing        | Vitest/Jest, Supertest, Playwright      | Unit/API/E2E testing                            |
| Deployment     | Vercel + Render/Railway + MongoDB Atlas | Production hosting                              |

## 9. System Architecture



## 10. Frontend Architecture

- Pages represent route-level screens.
- Reusable components handle UI elements.
- Hooks manage reusable state and Socket.IO subscriptions.
- Services contain API calls.
- Context can hold authenticated user/session state.
- Route guards enforce frontend navigation rules.
- Client-side validation improves UX; server-side validation remains mandatory.

```
client/src/
├── components/
├── pages/
├── layouts/
├── hooks/
├── services/
├── context/
├── routes/
├── utils/
├── assets/
└── App.jsx
```

## 11. Backend Architecture

```
server/
├── controllers/
├── routes/
├── models/
├── services/
├── middleware/
├── validators/
├── sockets/
├── utils/
├── config/
└── server.js
```

- Routes define API endpoints.
- Controllers handle request/response translation.
- Services contain queue business logic.
- Models contain Mongoose schemas/models.
- Middleware handles JWT verification, roles and validation.
- Socket handlers manage realtime events.
- Central error middleware returns consistent API errors.

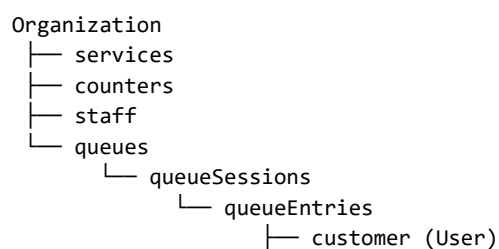
## 12. MongoDB Architecture

MongoDB will store application data as documents in collections. Mongoose will define schemas, validation rules, indexes and references. We will avoid blindly embedding large, frequently changing datasets; queue entries and users should remain independently queryable.

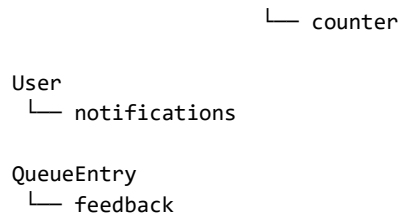
## 13. Collections and Schemas

| Collection    | Purpose                          | Important Fields                                                                                      |
|---------------|----------------------------------|-------------------------------------------------------------------------------------------------------|
| users         | All application identities       | _id, name, email, phone, passwordHash, role, status, createdAt                                        |
| organizations | Businesses/institutions          | _id, name, description, address, phone, email, logo, status                                           |
| services      | Services offered                 | _id, organizationId, name, description, averageServiceTime, priorityEnabled, status                   |
| counters      | Service counters                 | _id, organizationId, name, counterNumber, status                                                      |
| staff         | Staff assignments                | _id, userId, organizationId, counterId, status                                                        |
| queues        | Logical service queues           | _id, organizationId, serviceId, name, prefix, status, settings                                        |
| queueSessions | Daily/operational queue instance | _id, queueId, date, sequenceNumber, status, startedAt, endedAt                                        |
| queueEntries  | Customer tickets                 | _id, sessionId, queueId, customerId, tokenNumber, tokenLabel, status, priority, timestamps, counterId |
| notifications | User alerts                      | _id, userId, type, title, message, read, createdAt                                                    |
| feedback      | Post-service feedback            | _id, customerId, queueEntryId, rating, comment, createdAt                                             |
| workingHours  | Operating schedule               | _id, organizationId, day, openTime, closeTime, isOpen                                                 |

## 14. Relationships and References

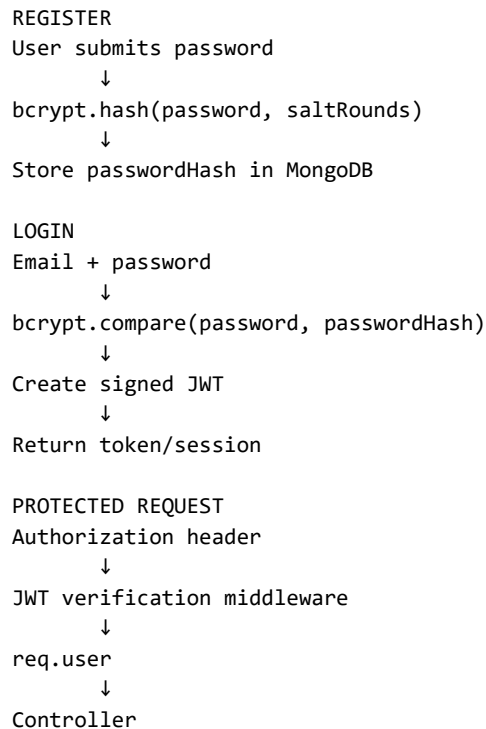






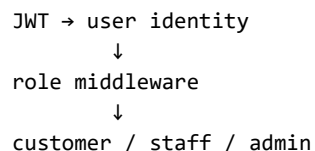
Use MongoDB ObjectId references through Mongoose where entities need independent lifecycle/querying. Do not duplicate mutable user or organization data inside every queue entry unless there is a deliberate snapshot requirement.

## 15. Authentication Architecture



- Never store plaintext passwords.
- Never store JWT secrets in frontend code.
- Use a strong JWT secret from environment variables.
- Set an appropriate token expiry.
- Consider httpOnly secure cookies for production authentication rather than exposing long-lived tokens to JavaScript.
- Validate email uniqueness at the database/application level.

## 16. Authorization and Role-Based Access



↓  
allowed controller operation

| Operation       | Customer | Staff       | Admin |
|-----------------|----------|-------------|-------|
| Join queue      | Yes      | Optional    | Yes   |
| View own ticket | Yes      | Yes         | Yes   |
| Call next       | No       | Yes         | Yes   |
| Skip/Recall     | No       | Yes         | Yes   |
| Mark served     | No       | Yes         | Yes   |
| Manage services | No       | No          | Yes   |
| Manage counters | No       | No          | Yes   |
| Manage staff    | No       | No          | Yes   |
| Analytics       | Own      | Queue-level | Full  |

## 17. Queue State Machine

| State     | Meaning                  | Allowed Next States                     |
|-----------|--------------------------|-----------------------------------------|
| waiting   | Customer waiting         | called, cancelled, skipped              |
| called    | Staff called customer    | serving, no_show, recalled              |
| serving   | Service in progress      | served                                  |
| served    | Completed                | terminal                                |
| skipped   | Skipped by staff         | terminal/recovery if explicitly allowed |
| cancelled | Customer left            | terminal                                |
| no_show   | Customer did not respond | terminal/recovery if policy allows      |

The server must reject invalid state transitions.

## 18. Token Generation

Token creation must happen on the server. The browser should never choose the next token number. Each queue session has its own sequence.

```
Queue prefix = A
Sequence = 105
```

```
New ticket:
tokenNumber = 105
tokenLabel  = A-105
```

- Create or identify the active queue session.
- Safely increment the session sequence.
- Create the queue entry.
- Return the generated ticket.
- Protect the operation against concurrent joins.

## 19. Queue Ordering and Position

For a basic queue, ordering is determined by priority and join time. The system should count only active entries in the same queue session.

```
ORDER BY
priority DESC,
joinedAt ASC
```

```
People Ahead =
active waiting entries
ahead of current ticket
within the same queue session
```

## 20. Estimated Wait Time

```
Basic MVP:
ETA = People Ahead × averageServiceTime
```

```
Example:
3 people ahead × 8 minutes
= approximately 24 minutes
```

Later, calculate rolling averages from actual service durations. This is standard statistics and does not require AI.

## 21. Priority Queue

- Priority must be assigned by an authorized workflow.
- Customers should not be able to grant themselves privileged priority.
- Priority ordering should be deterministic.
- Admin settings can define whether a queue supports priority.
- Priority rules should be documented per organization.

```
ORDER BY
priority DESC,
joinedAt ASC
```

## 22. Concurrency and Atomicity

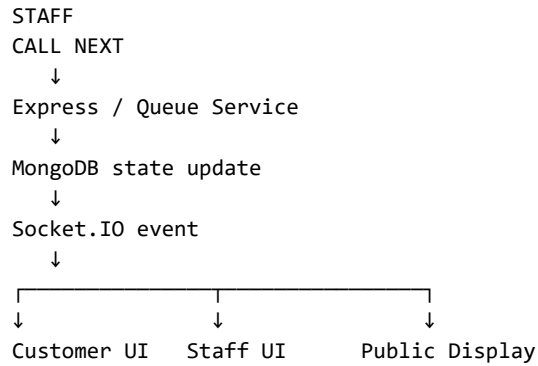
Concurrency is critical because two staff members may click CALL NEXT at nearly the same time, or two customers may join simultaneously. Queue-changing operations must use database-safe logic.

- Use MongoDB transactions where multiple documents must change together.
- Use conditional updates to ensure a ticket can only be transitioned from its expected current state.
- Use unique indexes for uniqueness requirements.
- Do not rely on frontend checks for duplicate prevention.
- Return a clear conflict response when another operation wins the race.

```
Example:
UPDATE/operation only if
status == "waiting"
```

```
If matchedCount == 0:
→ another operation already changed it
→ return conflict / retry safely
```

## 23. Socket.IO Real-Time Architecture



- Use rooms based on organization/queue/session IDs.
- Only authorized users should receive private ticket events.
- Public display events must contain only public-safe data.
- Reconnect clients and resync current state after connection loss.

## 24. API Specification

### Auth

POST /api/auth/register

POST /api/auth/login

POST /api/auth/logout

GET /api/auth/me

### Organizations

GET /api/organizations

GET /api/organizations/:id

POST /api/organizations

PUT /api/organizations/:id

DELETE /api/organizations/:id

### Services

GET /api/services

POST /api/services

PUT /api/services/:id

DELETE /api/services/:id

### Counters

GET /api/counters

POST /api/counters

PUT /api/counters/:id

DELETE /api/counters/:id

## Staff

GET /api/staff

POST /api/staff

PUT /api/staff/:id

DELETE /api/staff/:id

## Queues

GET /api/queues

POST /api/queues

GET /api/queues/:id

POST /api/queues/:id/join

POST /api/queues/:id/next

POST /api/queues/:id/pause

POST /api/queues/:id/resume

## Tickets

GET /api/tickets/:id

POST /api/tickets/:id/cancel

POST /api/tickets/:id/serve

POST /api/tickets/:id/skip

POST /api/tickets/:id/recall

POST /api/tickets/:id/no-show

## Notifications

GET /api/notifications

PATCH /api/notifications/:id/read

## Analytics

GET /api/analytics/today

GET /api/analytics/weekly

GET /api/analytics/queue/:id

# 25. UI / UX Screen Plan

| Screen  | Purpose                                            |
|---------|----------------------------------------------------|
| Landing | Explain QueueLess and allow organization discovery |

|                    |                                          |
|--------------------|------------------------------------------|
| Login/Register     | Authentication                           |
| Organizations      | Search and browse organizations          |
| Service Selection  | View service status and queue size       |
| Join Confirmation  | Confirm queue rules and join             |
| Live Ticket        | Token, position, ETA, counter and status |
| History            | Past queue tickets                       |
| Notifications      | Queue alerts                             |
| Staff Queue        | Operational queue control                |
| Admin Dashboard    | KPIs and charts                          |
| Service Management | Create/edit services                     |
| Counter Management | Manage counters                          |
| Staff Management   | Manage staff assignments                 |
| Queue Settings     | Prefix, capacity, average time, priority |
| Public Display     | Current token/counter                    |
| Kiosk              | Large touch interface                    |

## 26. Customer Dashboard

YOUR TOKEN

A-109

NOW SERVING

A-106

PEOPLE AHEAD

2

ESTIMATED WAIT

16 minutes

COUNTER

3

STATUS

WAITING

[LEAVE QUEUE]

## 27. Staff Dashboard

CURRENTLY SERVING

A-106

Counter 3

NEXT

A-107

A-108

A-109

[CALL NEXT]

[RECALL]

[SKIP]

[NO SHOW]

[MARK SERVED]

## 28. Admin Dashboard

- Total customers today
- Currently waiting
- Currently serving
- Completed
- Average wait time
- Average service time
- No-show rate
- Cancellation rate
- Peak hours
- Service/queue performance

|                   |        |
|-------------------|--------|
| Today's Customers | 248    |
| Currently Waiting | 37     |
| Served            | 191    |
| Avg Wait          | 14 min |

## 29. QR Queue Joining

```
QR
↓
https://queueless.app/join/{queueId}
↓
Queue information
↓
Login/identity
↓
Confirm
↓
Join
↓
Token
```

- QR should contain a non-sensitive public queue/service identifier.
- Do not place passwords, JWTs or private customer data inside the QR.
- QR flow should work well on mobile.

## 30. Kiosk Mode

WELCOME TO QUEUELESS

Select a Service

```
[ General ]
[ Billing ]
[ Registration ]
[ Support ]
```

↓

YOUR TOKEN  
A-108

The kiosk should use large buttons, minimal text and clear confirmation screens.

## 31. Public TV Display

NOW SERVING

A-104      Counter 1  
B-072      Counter 2  
A-105      Counter 3

NEXT

A-106  
A-107  
A-108

Never show customer phone numbers, email addresses or other private information on the public display.

## 32. Notifications

- Ticket created successfully.
- Position changed.
- Two/three customers ahead.
- Your turn is next.
- Your token is being called.
- Queue paused/resumed.
- Ticket cancelled/skipped.
- Service completed.

Start with in-app notifications. Email, browser push and SMS can be added later.

## 33. Validation and Error Handling

| Scenario                   | Expected Response                         |
|----------------------------|-------------------------------------------|
| Invalid login              | 401 Unauthorized with safe message        |
| Expired/invalid JWT        | 401 Unauthorized                          |
| Insufficient role          | 403 Forbidden                             |
| Invalid request data       | 400/422 validation response               |
| Queue closed               | Prevent join and show status              |
| Queue full                 | Prevent join                              |
| Duplicate active ticket    | Return existing ticket or block duplicate |
| Ticket not found           | 404                                       |
| Invalid state transition   | 409 Conflict                              |
| Concurrent queue operation | 409/retry-safe response                   |
| Realtime disconnect        | Reconnect and resync current state        |

## 34. Security

- Hash passwords using bcrypt.
- Keep JWT secret in environment variables.



- Never expose MongoDB credentials to the frontend.
- Validate all request payloads.
- Use authorization middleware on protected APIs.
- Sanitize/validate user-controlled strings as appropriate.
- Use HTTPS in production.
- Configure CORS for trusted frontend origins.
- Rate-limit authentication endpoints.
- Use MongoDB indexes and schema validation to protect data integrity.
- Do not return passwordHash in API responses.
- Log operational events without logging passwords or tokens.

## 35. Analytics

| Metric               | Definition                           |
|----------------------|--------------------------------------|
| Queue Volume         | Tickets created per day/hour/service |
| Average Wait         | Join → called                        |
| Average Service Time | Serving/call → served                |
| No-show Rate         | No-show / called                     |
| Cancellation Rate    | Cancelled / created                  |
| Completion Rate      | Served / created                     |
| Peak Hour            | Highest volume hour                  |
| Counter Utilization  | Service activity per counter         |

## 36. Project Folder Structure

```

QueueLess/
├── client/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── layouts/
│   │   ├── hooks/
│   │   ├── services/
│   │   ├── context/
│   │   ├── routes/
│   │   ├── utils/
│   │   └── App.jsx
│   └── package.json
├── server/
│   ├── controllers/
│   ├── routes/
│   ├── models/
│   ├── services/
│   ├── middleware/
│   ├── validators/
│   ├── sockets/
│   ├── utils/
│   ├── config/
│   └── server.js
└── tests/

```

```
| .env.example
| .gitignore
| README.md
```

## 37. Development Roadmap

| Phase | Focus           | Output                                              |
|-------|-----------------|-----------------------------------------------------|
| 1     | Project setup   | React/Vite + Express + Git                          |
| 2     | MongoDB Atlas   | Connection + Mongoose base setup                    |
| 3     | Authentication  | JWT + bcrypt + protected routes                     |
| 4     | Database models | Users, organizations, services, counters, queues    |
| 5     | Admin           | Organization/service/counter/staff management       |
| 6     | Queue engine    | Sessions, tokens, join, position, state transitions |
| 7     | Customer        | Live ticket, history, cancellation                  |
| 8     | Staff           | Call/serve/skip/recall/no-show                      |
| 9     | Socket.IO       | Realtime queue synchronization                      |
| 10    | Notifications   | In-app queue alerts                                 |
| 11    | Analytics       | Admin metrics and charts                            |
| 12    | QR              | QR joining                                          |
| 13    | Kiosk/display   | On-site kiosk and TV display                        |
| 14    | Testing         | Unit/API/E2E/concurrency                            |
| 15    | Deployment      | Vercel + Render/Railway + Atlas                     |
| 16    | Polish          | Responsive UI, errors, loading states, docs         |

## 38. Testing Strategy

- Unit test token generation and queue-order calculations.
- Test JWT authentication and password verification.
- API-test all protected role-specific endpoints.
- Test invalid state transitions.
- Test duplicate active ticket prevention.
- Test concurrent join/call operations.
- Test Socket.IO updates across multiple clients.
- Test customer join → staff call → serve → feedback end-to-end.
- Test mobile/tablet/desktop responsiveness.
- Test reconnect and resync behavior.

Critical E2E:

Customer joins

- token created
- staff calls next
- customer updates live
- staff serves
- ticket completed
- customer submits feedback

## 39. Deployment

47. Create MongoDB Atlas production cluster/database.
48. Create database user with appropriate permissions.
49. Configure Network Access securely.
50. Deploy Express/Socket.IO backend to Render/Railway.
51. Set production environment variables.
52. Deploy React frontend to Vercel.
53. Configure API base URL.
54. Configure CORS for production frontend.
55. Test Socket.IO through production deployment.
56. Run authentication and queue smoke tests.
57. Add custom domain if desired.
58. Monitor logs and errors.

## 40. Environment Variables

```
SERVER
PORT=
MONGODB_URI=
JWT_SECRET=
JWT_EXPIRES_IN=
CLIENT_URL=
NODE_ENV=

CLIENT
VITE_API_URL=
VITE_SOCKET_URL=
```

Do not commit real .env files. Commit only .env.example with placeholder names.

## 41. Git Strategy

```
main
├── develop
│   ├── feature/auth
│   ├── feature/mongodb-models
│   ├── feature/queue-engine
│   ├── feature/customer-dashboard
│   ├── feature/staff-dashboard
│   ├── feature/admin-dashboard
│   └── feature/realtime
```

- Keep main stable.
- Use feature branches.
- Make focused commits.
- Never commit .env, node\_modules or secrets.
- Keep README updated.

## 42. MVP vs Advanced Features

| MVP                            | Advanced                                          |
|--------------------------------|---------------------------------------------------|
| JWT + bcrypt authentication    | Refresh-token rotation / hardened cookie strategy |
| Customer/Staff/Admin roles     | Fine-grained permission matrix                    |
| Organizations/services/queues  | Multi-branch organizations                        |
| Token generation               | Advanced priority policies                        |
| Position + basic ETA           | Rolling service-time analytics                    |
| Call/serve/skip/recall/no-show | Complex service rules                             |
| Socket.IO realtime             | Offline-friendly reconnect handling               |
| Basic notifications            | Browser/email/SMS notifications                   |
| Basic analytics                | Advanced exports/reporting                        |
| QR joining                     | Printed kiosk ticketing                           |
| Customer feedback              | Detailed survey analytics                         |
| Staff dashboard                | Advanced counter balancing                        |

## 43. Resume / Portfolio Positioning

### QueueLess — Real-Time Virtual Queue Management Platform

- Built a full-stack virtual queue management platform using React, Node.js/Express and MongoDB Atlas.
- Implemented JWT + bcrypt authentication with role-based access for customers, staff and administrators.
- Designed queue state transitions and concurrency-safe token operations using MongoDB/Mongoose.
- Implemented real-time queue synchronization using Socket.IO for customer, staff and public display interfaces.
- Developed operational dashboards covering queue volume, waiting time, service time and completion metrics.
- Added QR-based queue joining and designed kiosk/public-display workflows for real-world deployment.

## 44. Future Scope

- Multi-branch support.
- Appointment + virtual queue hybrid.
- Email/browser/SMS notifications.
- Payments for paid services.
- Multi-language UI.
- Accessibility improvements.
- Offline-friendly kiosk.
- Advanced SLA and capacity rules.
- CSV/PDF reporting.
- Public organization directory.
- Conventional statistical forecasting if needed, without AI.

## 45. Definition of Done

- Customer can register/login securely.
- Admin can create and manage organization data.

- Admin can manage services, counters, staff and queues.
- Customer can join an active queue and receive a unique token.
- Position and ETA are correct for active queue ordering.
- Staff can call, recall, skip, no-show and serve tickets.
- Concurrent operations cannot accidentally serve/call the same ticket twice.
- Socket.IO updates connected clients in real time.
- Customer can cancel an active ticket.
- Queue history persists in MongoDB.
- Notifications work for important queue events.
- Analytics work for core operational metrics.
- QR joining works.
- Application is responsive.
- Secrets are protected.
- Production deployment works.
- End-to-end customer-to-service flow passes successfully.

## 46. Exact Build Order for Antigravity

Antigravity must treat this document as the master specification. Do not implement the entire application in one step. Build and test each phase before moving to the next.

59. Read and analyze this specification.
60. Confirm the machine has Node.js, npm and Git; do not reinstall existing tools unnecessarily.
61. Create the React/Vite client and Node/Express server.
62. Create the MongoDB Atlas database and secure connection.
63. Create Mongoose models and indexes.
64. Implement JWT + bcrypt authentication.
65. Implement role-based authorization middleware.
66. Build admin organization/service/counter/staff management.
67. Implement queue sessions and safe token generation.
68. Implement customer join/cancel/ticket flow.
69. Implement position and ETA calculations.
70. Build staff queue-control operations.
71. Implement valid queue state transitions.
72. Add Socket.IO rooms and realtime events.
73. Build customer live updates.
74. Build staff live updates.
75. Build admin analytics.
76. Add notifications.
77. Add QR queue joining.
78. Add kiosk mode.
79. Add public TV display.
80. Write unit/API/E2E/concurrency tests.
81. Deploy backend and Socket.IO.
82. Deploy frontend.
83. Configure production MongoDB Atlas.
84. Run complete production smoke test.
85. Prepare README, screenshots, architecture diagram and demo.

### CRITICAL INSTRUCTION FOR THE IDE

Do not replace MongoDB with PostgreSQL, Supabase or Firebase.  
Do not replace JWT + bcrypt with another authentication system without explicit approval.  
Do not replace Socket.IO with Supabase Realtime.  
Do not add AI features.  
Do not build everything at once.  
Keep database models, API contracts, frontend and realtime events synchronized.

## 47. First Antigravity Prompt

Use the following prompt immediately after uploading this document to Antigravity:

I have uploaded the QueueLess Final Master Specification.

Treat this document as the SINGLE SOURCE OF TRUTH for the project.

First, do NOT implement the application.

QueueLess — Final Master Specification | MongoDB + JWT + Socket.IO

Analyze the specification and report:

1. Final architecture
2. Required tools and versions
3. Existing tools that need to be checked on my machine
4. Complete folder structure
5. MongoDB collections and Mongoose models
6. Authentication architecture using JWT + bcrypt
7. Socket.IO architecture
8. Required npm packages
9. Environment variables
10. Development phases and dependencies between phases

Important:

- Use MongoDB Atlas + Mongoose.
- Use JWT + bcrypt authentication.
- Use Socket.IO for realtime.
- Use React + Vite + Tailwind CSS on frontend.
- Use Node.js + Express.js on backend.
- Do not use PostgreSQL, Supabase, Firebase or AI.
- Do not reinstall software that is already installed.
- Do not start coding yet.
- Wait for my confirmation before Phase 1.

After Antigravity provides the analysis, start with Phase 1 only. Do not allow it to silently redesign the architecture.